

Lab_Regressao_California_Real_Devs

September 1, 2026

1 Laboratório: Regressão Linear

Objetivo: Sair das simulações numéricas e aplicar os conceitos dos slides em **dados reais**.

Vamos utilizar o famoso dataset **California Housing** (Preços de imóveis na Califórnia), que contém ruídos, *outliers*, problemas de escala e comportamentos não-lineares.

```
[2]: # Importações essenciais
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import scipy.stats as stats
from sklearn.datasets import fetch_california_housing
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import r2_score

# Configuração visual profissional
sns.set_theme(style='whitegrid')
plt.rcParams['figure.figsize'] = (10, 6)
```

1.1 1. Conhecendo o nosso Banco de Dados

Vamos importar os dados diretamente do Scikit-Learn. Nossos atributos (X) incluem renda média do bairro, idade das casas, número de cômodos, população, etc. Nosso alvo (Y) é o valor mediano da casa (em centenas de milhares de dólares).

```
[20]: # Carregando o dataset real
california = fetch_california_housing(as_frame=True)
df = california.frame

# Visualizando as primeiras linhas do DataFrame
display(df.head())

# Separando X (Features) e y (Alvo)
X_real = df.drop(columns=['MedHouseVal'])
y_real = df['MedHouseVal']
```

```
print(f"Total de registros (imóveis): {df.shape[0]}")
print(f"Total de atributos: {X_real.shape[1]}")
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	\
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	

	Longitude	MedHouseVal
0	-122.23	4.526
1	-122.22	3.585
2	-122.24	3.521
3	-122.25	3.413
4	-122.25	3.422

Total de registros (imóveis): 20640

Total de atributos: 8

1.2 2. Mínimos Quadrados com Álgebra Linear (Equação Normal)

Vamos provar que a equação $\hat{\beta} = (X^T X)^{-1} X^T Y$ funciona com dados reais. Para o processamento matricial não demorar e não sofrer com a falta de normalização extrema, vamos pegar uma amostra de 1000 linhas e apenas 2 variáveis: Renda Média (MedInc) e Idade da Casa (HouseAge).

```
[16]: # =====
# IMPORTAÇÕES E CONFIGURAÇÕES
# =====

import warnings
# Suprimindo alertas chatos de bibliotecas de terceiros (DeprecationWarnings)
warnings.filterwarnings("ignore", category=DeprecationWarning)

# Importando o Plotly para gráficos interativos (padrão de mercado para
↳ Dashboards)
import plotly.graph_objects as go
import numpy as np

# 1. Selecionando amostra e features para o teste matricial
amostra = df.sample(1000, random_state=42)
X_amostra = amostra[['MedInc', 'HouseAge']].values
y_amostra = amostra['MedHouseVal'].values.reshape(-1, 1)

# 2. Adicionando a coluna de '1's para calcular o Intercepto (Viés)
X_matriz = np.c_[np.ones((X_amostra.shape[0], 1)), X_amostra]

# 3. O Treinamento Matemático usando o operador '@' do Python
pesos_beta = np.linalg.inv(X_matriz.T @ X_matriz) @ X_matriz.T @ y_amostra
```

```

vies = pesos_beta[0][0]
peso_renda = pesos_beta[1][0]
peso_idade = pesos_beta[2][0]

print("--- PESOS CALCULADOS MANUALMENTE VIA ÁLGEBRA LINEAR ---")
print(f"Intercepto (Preço Base): {vies:.4f}")
print(f"Peso da Renda: {peso_renda:.4f}")
print(f"Peso da Idade: {peso_idade:.4f}")

# A. Criando a nuvem de pontos (Casas Reais)
scatter = go.Scatter3d(
    x=X_amostra[:, 0],
    y=X_amostra[:, 1],
    z=y_amostra.flatten(),
    mode='markers',
    marker=dict(size=4, color='blue', opacity=0.5),
    name='Casas Reais'
)

# B. Criando a malha do plano matemático
x_surf = np.linspace(X_amostra[:, 0].min(), X_amostra[:, 0].max(), 20)
y_surf = np.linspace(X_amostra[:, 1].min(), X_amostra[:, 1].max(), 20)
x_surf, y_surf = np.meshgrid(x_surf, y_surf)

# C. Calculando o Z do plano ( $Z = Viés + W1*X1 + W2*X2$ )
z_surf = vies + (peso_renda * x_surf) + (peso_idade * y_surf)

plano = go.Surface(
    x=x_surf,
    y=y_surf,
    z=z_surf,
    colorscale='Reds',
    opacity=0.7,
    showscale=False,
    name='Modelo (Plano Matemático)'
)

# D. Montando e renderizando o gráfico interativo
fig = go.Figure(data=[scatter, plano])
fig.update_layout(
    title='A Matemática em Ação: Gire o Gráfico com o Mouse!',
    scene=dict(
        xaxis_title='Renda Média',
        yaxis_title='Idade da Casa',
        zaxis_title='Preço (100k USD)'
    ),
)

```

```

        width=900,
        height=700,
        margin=dict(l=0, r=0, b=0, t=40)
    )
    fig.show()

```

--- PESOS CALCULADOS MANUALMENTE VIA ÁLGEBRA LINEAR ---

Intercepto (Preço Base): 0.0311

Peso da Renda: 0.4065

Peso da Idade: 0.0161

```

[17]: # A. Criando a nuvem de pontos (Casas Reais)
scatter = go.Scatter3d(
    x=X_amostra[:, 0],
    y=X_amostra[:, 1],
    z=y_amostra.flatten(),
    mode='markers',
    marker=dict(size=4, color='blue', opacity=0.7),
    name='Casas Reais (Y)'
)

# B. Criando a malha do plano matemático
x_surf = np.linspace(X_amostra[:, 0].min(), X_amostra[:, 0].max(), 20)
y_surf = np.linspace(X_amostra[:, 1].min(), X_amostra[:, 1].max(), 20)
x_surf, y_surf = np.meshgrid(x_surf, y_surf)
z_surf = vies + (peso_renda * x_surf) + (peso_idade * y_surf)

plano = go.Surface(
    x=x_surf, y=y_surf, z=z_surf,
    colorscale='Reds', opacity=0.5, showscale=False,
    name='Previsão do Sistema'
)

# C. DESENHANDO OS ERROS (Os "Fios de Prumo")
# Vamos calcular onde cada casa real "bate" no plano
z_pred_amostra = vies + (peso_renda * X_amostra[:, 0]) + (peso_idade *
↪ X_amostra[:, 1])

# Criando as linhas verticais do ponto real até o plano
linhas_x, linhas_y, linhas_z = [], [], []

# Para o gráfico não ficar poluído, vamos desenhar o erro só de 50 casas
↪ aleatórias
for i in range(50):
    linhas_x.extend([X_amostra[i, 0], X_amostra[i, 0], None])
    linhas_y.extend([X_amostra[i, 1], X_amostra[i, 1], None])
    linhas_z.extend([y_amostra[i][0], z_pred_amostra[i], None])

```

```

erros_visuais = go.Scatter3d(
    x=linhas_x, y=linhas_y, z=linhas_z,
    mode='lines',
    line=dict(color='black', width=3, dash='dot'),
    name='Resíduos (Erros minimizados)'
)

# D. Montando e renderizando o gráfico interativo
fig = go.Figure(data=[scatter, plano, erros_visuais])
fig.update_layout(
    title='Entendendo o Melhor Ajuste: As linhas pontilhadas são os Erros!',
    scene=dict(
        xaxis_title='Renda Média',
        yaxis_title='Idade da Casa',
        zaxis_title='Preço (100k USD)'
    ),
    width=900, height=700, margin=dict(l=0, r=0, b=0, t=40)
)
fig.show()

```

1.3 3. O Modo Desenvolvedor: Scikit-Learn e R^2 Ajustado

Agora vamos treinar o modelo na base completa usando a biblioteca padrão de mercado e avaliar sua nota.

```

[21]: # Instanciando e treinando o modelo com todas as features
modelo_dev = LinearRegression()
modelo_dev.fit(X_real, y_real)

# Previsões na base toda
previsoes = modelo_dev.predict(X_real)

# Calculando as métricas
r2_tradicional = r2_score(y_real, previsoes)
N, p = X_real.shape
r2_ajustado = 1 - (1 - r2_tradicional) * (N - 1) / (N - p - 1)

print("===== AVALIAÇÃO DO SISTEMA =====")
print(f"R-Quadrado (Acurácia Linear): {r2_tradicional:.4f}")
print(f"R-Quadrado Ajustado: {r2_ajustado:.4f}")
print("=====\\n")

print("Impacto de cada variável no preço da casa (Pesos/Coefficientes):")
pesos_df = pd.DataFrame({
    'Feature (Entrada)': X_real.columns,
    'Peso Aprendido (w)': modelo_dev.coef_
})

```

```

}).sort_values(by='Peso Aprendido (w)', ascending=False)

display(pesos_df)

plt.figure(figsize=(10, 6))
# Plotando Real vs Previsto
plt.scatter(y_real, previsoes, color='teal', alpha=0.3, label='Previsões do_
↳Sistema')

# Linha de Perfeição (45 graus - onde Y_real == Y_previsto)
min_val = min(y_real.min(), previsoes.min())
max_val = max(y_real.max(), previsoes.max())
plt.plot([min_val, max_val], [min_val, max_val], color='red', linewidth=3,
↳linestyle='-', label='Linha de Acerto Perfeito (R² = 1.0)')

plt.title('Realidade vs Expectativa: Quão boas são as previsões?')
plt.xlabel('Preço REAL (100k USD)')
plt.ylabel('Preço PREVISTO pela IA (100k USD)')
plt.legend()

# Anotação didática
plt.annotate('Se o modelo fosse perfeito,\ntodas as bolinhas estariam\nem cima_
↳da linha vermelha.',
            xy=(2, 2), xytext=(0, 4), fontsize=11,
            bbox=dict(boxstyle="round", alpha=0.1, color='red'),
            arrowprops=dict(facecolor='red', shrink=0.05))

plt.show()

```

```

===== AVALIAÇÃO DO SISTEMA =====
R-Quadrado (Acurácia Linear): 0.6062
R-Quadrado Ajustado:          0.6061
=====

```

Impacto de cada variável no preço da casa (Pesos/Coefficientes):

	Feature (Entrada)	Peso Aprendido (w)
3	AveBedrms	0.645066
0	MedInc	0.436693
1	HouseAge	0.009436
4	Population	-0.000004
5	AveOccup	-0.003787
2	AveRooms	-0.107322
6	Latitude	-0.421314
7	Longitude	-0.434514



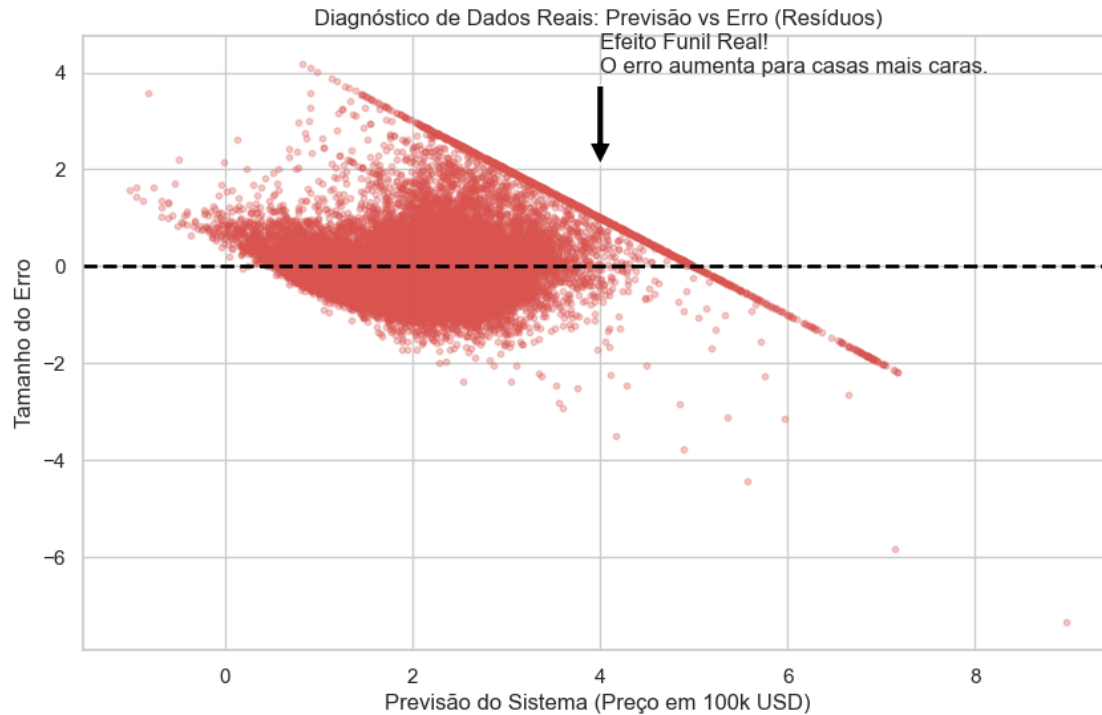
1.4 4. Debugging 1 e 2: O Gráfico de Resíduos e Heterocedasticidade

Lembram do “Efeito Funil” dos slides? Quando aplicamos Regressão Linear em preços reais de casas, os imóveis baratos costumam ter pouco erro, mas nas mansões, o erro “explode”. Vamos ver isso acontecendo.

```
[22]: # Calculando o Erro (Resíduo = Real - Previsto)
erros_reais = y_real - previsoes

plt.scatter(previsoes, erros_reais, color='#D9534F', alpha=0.3, s=10)
plt.axhline(0, color='black', linestyle='--', linewidth=2)
plt.title('Diagnóstico de Dados Reais: Previsão vs Erro (Resíduos)')
plt.xlabel('Previsão do Sistema (Preço em 100k USD)')
plt.ylabel('Tamanho do Erro')
plt.annotate('Efeito Funil Real! \n0 erro aumenta para casas mais caras.',
            xy=(4, 2), xytext=(4, 4), fontsize=12,
            arrowprops=dict(facecolor='black', shrink=0.05))

# Um detalhe curioso dos dados reais: note uma linha oblíqua e um limite
# superior.
# Isso acontece porque o governo cortou (clipping) o valor das casas nos dados
# reais.
plt.show()
```



1.5 5. Debugging 3: O Gráfico Q-Q (As caudas pesadas)

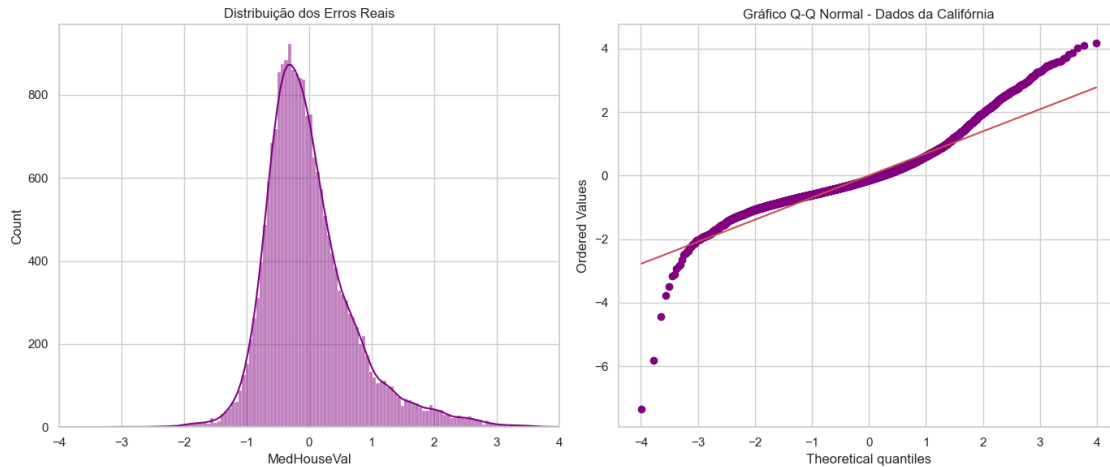
Será que os erros da previsão de imóveis seguem uma curva de sino perfeita (Normal)? Vamos testar no Q-Q Plot. Observe as pontas (caudas) se distanciando da reta: isso indica outliers de preços na vida real.

```
[23]: fig, ax = plt.subplots(1, 2, figsize=(14, 6))

sns.histplot(erros_reais, kde=True, ax=ax[0], color='purple')
ax[0].set_title('Distribuição dos Erros Reais')
ax[0].set_xlim(-4, 4)

stats.probplot(erros_reais, dist="norm", plot=ax[1])
ax[1].set_title('Gráfico Q-Q Normal - Dados da Califórnia')
ax[1].get_lines()[0].set_markerfacecolor('purple')
ax[1].get_lines()[0].set_markeredgecolor('purple')

plt.tight_layout()
plt.show()
print("As pontas fogem muito da linha vermelha. Na vida real, preços de casas_
    ↪ têm 'caudas pesadas' (muitos outliers de luxo e anomalias).")
```

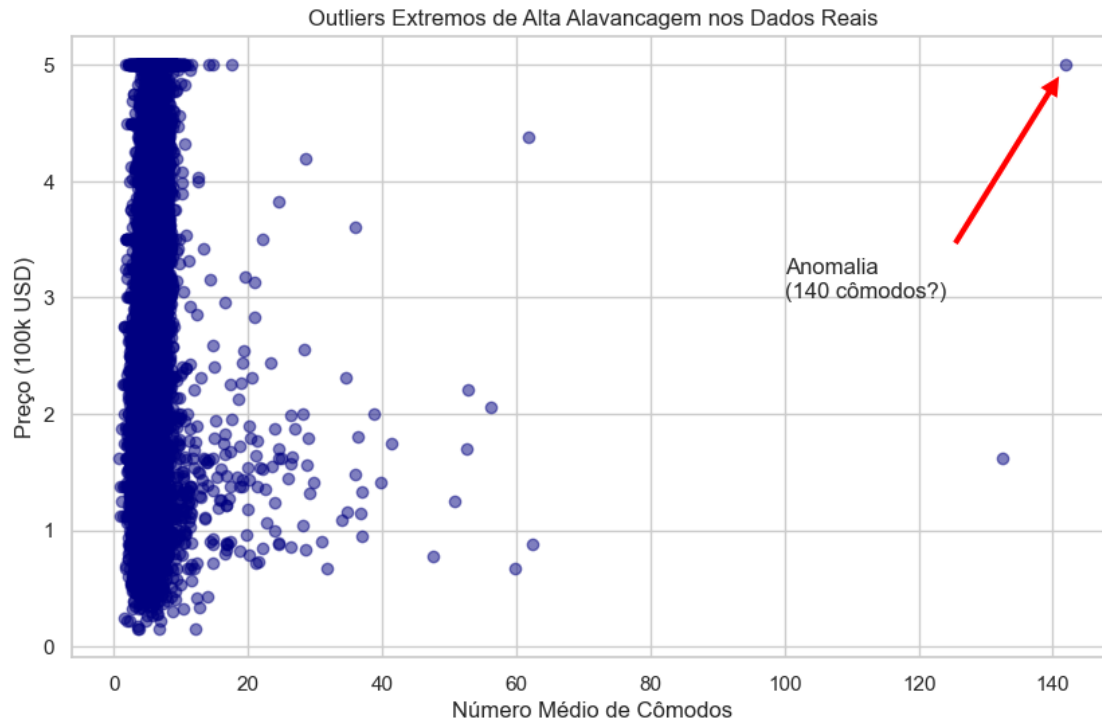


As pontas fogem muito da linha vermelha. Na vida real, preços de casas têm 'caudas pesadas' (muitos outliers de luxo e anomalias).

1.6 6. O Efeito Alavanca: O Perigo dos Outliers no Banco de Dados

Vamos cruzar o preço da casa com o 'Número Médio de Cômodos' (AveRooms). Na vida real, a maioria das casas tem 3 a 8 cômodos. Mas há registros bizarros de imóveis com mais de 130 cômodos no dataset (provavelmente hotéis registrados errado ou resorts). Veja como eles prejudicam a visualização.

```
[24]: plt.scatter(df['AveRooms'], df['MedHouseVal'], color='navy', alpha=0.5)
plt.title('Outliers Extremos de Alta Alavancagem nos Dados Reais')
plt.xlabel('Número Médio de Cômodos')
plt.ylabel('Preço (100k USD)')
plt.annotate('Anomalia \n(140 cômodos?)',
             xy=(142, 5), xytext=(100, 3), fontsize=12,
             arrowprops=dict(facecolor='red', shrink=0.05))
plt.show()
print("Se rodarmos o .fit() com esse imóvel de 140 cômodos, ele agirá como um
↪ímã (Pivô), distorcendo toda a nossa equação e afetando as previsões de
↪clientes comuns.")
```



Se rodarmos o `.fit()` com esse imóvel de 140 cômodos, ele agirá como um ímã (Pivô), distorcendo toda a nossa equação e afetando as previsões de clientes comuns.

1.7 7. Refatoração de Dados: StandardScaler

Em bancos de dados reais, as unidades são caóticas. - `MedInc` varia de 0.5 a 15. - `Population` varia de 3 a 35.000. Multiplicar matrizes com essa disparidade pode causar erro de processamento na CPU (*Overflow*). Vamos padronizar.

```
[25]: # Verificando a bagunça de escalas original
print("--- MÉDIAS ANTES DA PADRONIZAÇÃO ---")
print(X_real[['MedInc', 'Population']].mean())

# Aplicando StandardScaler
scaler = StandardScaler()
X_escalonado = scaler.fit_transform(X_real)

# Convertendo de volta para DataFrame para visualizar
X_escalonado_df = pd.DataFrame(X_escalonado, columns=X_real.columns)

print("\n--- MÉDIAS DEPOIS DA PADRONIZAÇÃO (Praticamente Zero) ---")
print(X_escalonado_df[['MedInc', 'Population']].mean())
```

```
print("\n--- DESVIO PADRÃO DEPOIS DA PADRONIZAÇÃO (Exatamente 1) ---")
print(X_escalonado_df[['MedInc', 'Population']].std())

print("\n O treinamento será mais rápido e numericamente estável!")
```

--- MÉDIAS ANTES DA PADRONIZAÇÃO ---

```
MedInc      3.870671
Population  1425.476744
dtype: float64
```

--- MÉDIAS DEPOIS DA PADRONIZAÇÃO (Praticamente Zero) ---

```
MedInc      6.609700e-17
Population  -1.101617e-17
dtype: float64
```

--- DESVIO PADRÃO DEPOIS DA PADRONIZAÇÃO (Exatamente 1) ---

```
MedInc      1.000024
Population  1.000024
dtype: float64
```

O treinamento será mais rápido e numericamente estável!